```java

import java.util.*;

class Item {
    int profit, weight;
    double ratio;

    Item(int profit, int weight) {
        this.profit = profit;
        this.weight = weight;
        this.ratio = (double) profit / weight;
    }
}

public class FractionalKnapsack {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Number of items: ");
        int n = sc.nextInt();

        if (n <= 0) {
            System.out.println("Invalid input: At least one item is required.");
            sc.close();
            return;
        }

        Item[] items = new Item[n];

        for (int i = 0; i < n; i++) {
            System.out.print("Profit and weight of item " + (i + 1) + ": ");
            int profit = sc.nextInt();
            int weight = sc.nextInt();

            if (profit < 0 || weight <= 0) {
                System.out.println("Invalid item data: Profit must be non-
negative and weight must be positive.");
                sc.close();
                return;
            }

            items[i] = new Item(profit, weight);
        }

        System.out.print("Knapsack capacity: ");
        int capacity = sc.nextInt();
```

```
46
47          if (capacity < 0) {
48              System.out.println("Invalid capacity: Knapsack capacity cannot be negative.");
49              sc.close();
50              return;
51          }
52
53          if (capacity == 0) {
54              System.out.println("Maximum profit: 0.00");
55              System.out.println("No items can be selected because the knapsack has zero capacity.");
56              sc.close();
57              return;
58          }
59
60          Arrays.sort(items, (a, b) ->
61              Double.compare(b.ratio, a.ratio)
62          );
63
64          double totalProfit = 0;
65          int remaining = capacity;
66
67          for (int i = 0; i < n && remaining > 0; i++) {
68              if (items[i].weight <= remaining) {
69                  remaining -= items[i].weight;
70                  totalProfit += items[i].profit;
71              } else {
72                  totalProfit += items[i].profit *
73                      ((double) remaining / items[i].weight);
74                  remaining = 0;
75              }
76          }
77
78          System.out.printf("Maximum profit using the Greedy method: %.2f%n",
79                  totalProfit);
80
81          if (remaining == 0) {
82              System.out.println("Knapsack capacity utilized completely.");
83          } else {
84              System.out.println("Knapsack capacity utilized partially.");
85          }
86
87          sc.close();
88      }
89  }
90
```

Output :

```
PS D:\Assignment_03_Fractional_Knapsack_Greedy> javac FractionalKnapsack.java
PS D:\Assignment_03_Fractional_Knapsack_Greedy> java FractionalKnapsack
Number of items: 3
Profit and weight of item 1: 60 10
Profit and weight of item 2: 100 20
Profit and weight of item 3: 120 30
Knapsack capacity: 50
Maximum profit using the Greedy method: 240.00
Knapsack capacity utilized completely.
PS D:\Assignment_03_Fractional_Knapsack_Greedy>
```